

PARADIGMAS DE PROGRAMACIÓN
CURSO SUAREZ

Trabajo Práctico 1 Programación Orientada a Objetos

× ×

25 de septiembre de 2025

Alexander Cruz
109044
acruz@fi.uba.ar

Índice

1. Introducción	3
2. Objetivo	3
3. Consigna	3
4. Alcance	3
5. Detalles de Implementación	4
6. Diagrama de Clases	5
7. Diagrama de Secuencias	6
8. Cobertura	6

1. Introducción

El presente informe detalla la solución desarrollada para el Trabajo Práctico 1, que tiene como objetivo aplicar los conceptos fundamentales de la programación orientada a objetos (POO) en el lenguaje Pharo. El problema, que consiste en el desarrollo de un sistema de recomendación de futbolistas, fue resuelto utilizando la metodología Test-Driven Development (TDD), un enfoque que garantiza la robustez y funcionalidad del código a través de pruebas unitarias.

La solución se centra en la construcción de un modelo de clases que se adhiere a los principios de la POO, como el encapsulamiento y, de manera crucial, el polimorfismo. Se ha diseñado una arquitectura que permite manejar la lógica de cálculo del valor de los futbolistas y los distintos criterios de búsqueda de manera flexible y extensible, evitando el uso de estructuras de control rígidas.

Adicionalmente, se ha utilizado la herramienta Mutalk para llevar a cabo una prueba de mutación. Este proceso aseguró la calidad de las pruebas unitarias creadas por el alumno, garantizando que el código no solo funciona, sino que está correctamente validado contra posibles errores. El informe incluye los diagramas de clases y de secuencia que documentan el diseño, junto con una explicación detallada de la implementación y los mecanismos de polimorfismo utilizados.

2. Objetivo

Este es un trabajo práctico individual que tiene como objetivo aplicar los conceptos del paradigma de la orientación a objetos vistos hasta ahora en el curso mediante la resolución de un problema con Pharo.

3. Consigna

Se deberá desarrollar un modelo de clases utilizando la metodología TDD. Las especificaciones son un archivo de pruebas SUnit (.st) adjunto a este enunciado y están dadas en forma de pruebas de casos de uso. Se espera que el alumno genere una solución que permita ejecutar exitosamente las pruebas especificadas manteniendo una alta cobertura y evitando la supervivencia de mutantes. Revisar el material Calidad de nuestras pruebas.

4. Alcance

Se deberá desarrollar el modelo completo de la aplicación, sin incluir la interfaz gráfica. Para esto se especifica una API que está descrita por un conjunto de casos de uso. Este conjunto de pruebas está descrito por la clase dada como enunciado.

Para el manejo de errores y situaciones de excepción, se deberán desarrollar y utilizar las excepciones que se crean convenientes, más allá de que en la prueba de integración provistas no necesariamente se manejan excepciones.

Deberán entregarse:

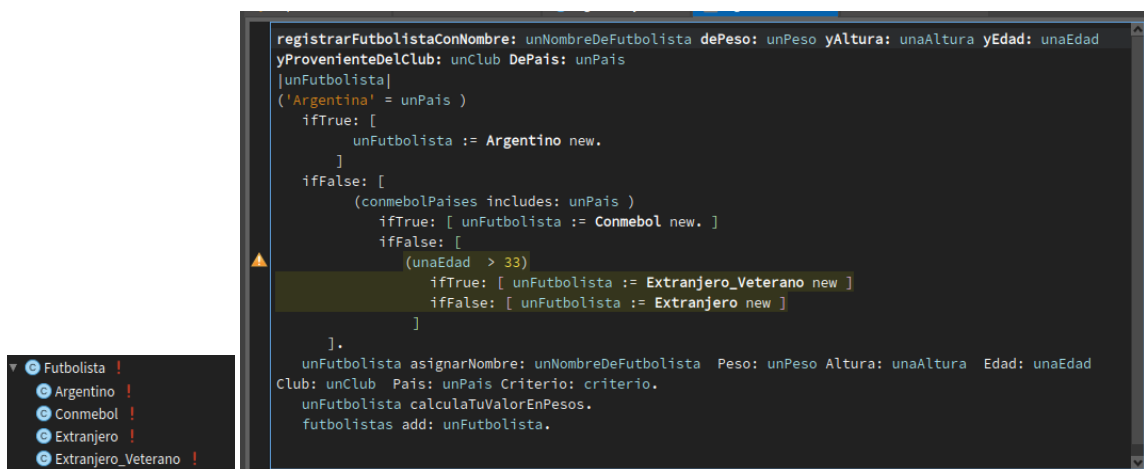
- todas las clases con sus métodos que resuelven el problema, organizados en una o más categorías según criterio del alumno;
- conjunto de pruebas que garanticen la no supervivencia de mutantes;
- conjunto de pruebas que garanticen la no supervivencia de mutantes;
- un archivo ".txt" que contenga la pieza de código que permite ejecutar los test de mutación (descrito aquí: pruebas de mutación);

documentación completa del diseño de clases incluyendo diagramas UML de clases y secuencia (cantidades variables acordes al diseño presentado). Se considerarán entregas válidas a aquellas que contengan las partes anteriores. Las que no tengan alguna de ellas no recibirán corrección.

5. Detalles de Implementación

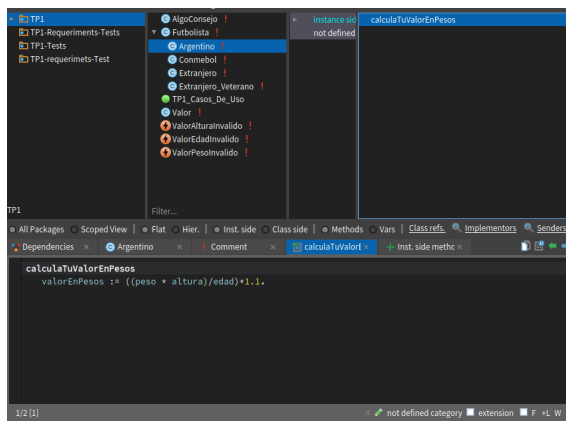
El Polimorfismo, una de las piedras angulares de la Programación Orientada a Objetos (POO), se implementó de manera central en la lógica de cálculo del valor de los futbolistas. Este concepto nos permitió tratar a todos los objetos de tipo Futbolista de manera uniforme, mientras que cada objeto ejecutaba su propia lógica específica .

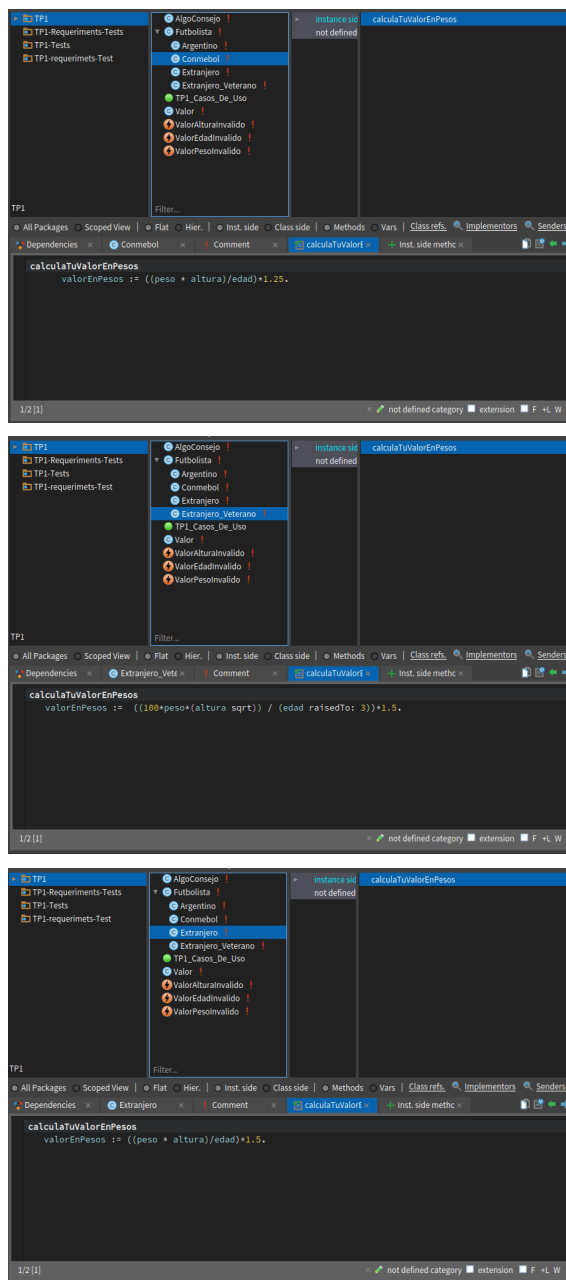
Cada objeto pudo responder el mensaje `calculaTuValorEnPesos()`, esto se logro debido a que se uso herencia, respetando la regla de .es un”, ademas se tuvo en cuenta las consecuencias de aplicar herencia , pero en estos ejemplos concretos no afectan el funcionamiento.



El metodo me permite enviar el mismo mensaje para las 4 instancias de objeto.

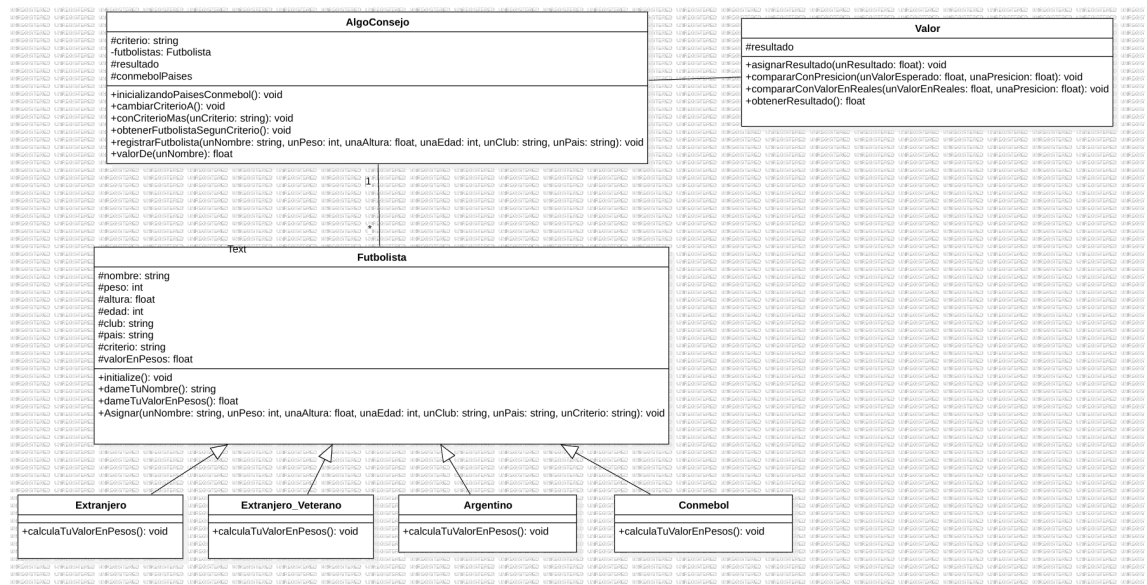
Ademas se observan las siguientes implementaciones para cada una de las clases derivadas de futbolista.





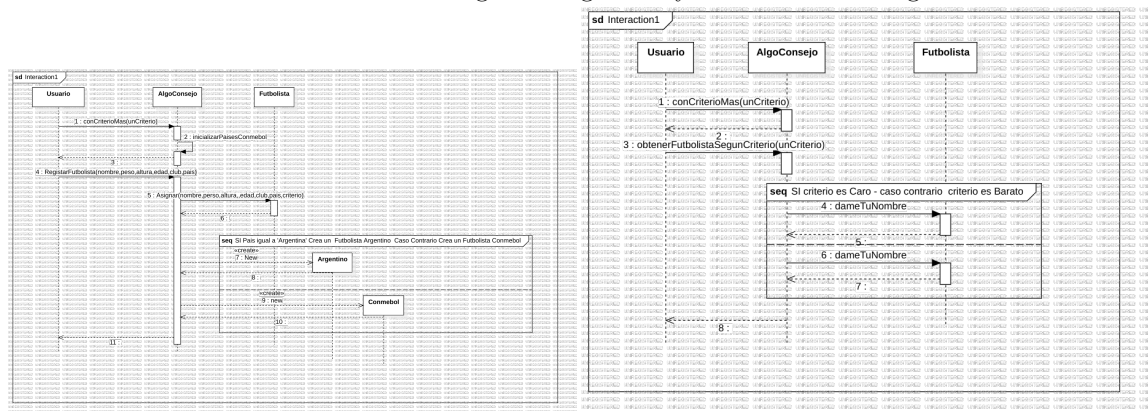
6. Diagrama de Clases

Se usaron un total de 7 diagramas de clases para presentar todo el proyecto, con las respectivas asociaciones y herencia.



7. Diagrama de Secuencias

Los diagramas de secuencia me permiten vizualizar el seguimiento de dos situaciones particulares: Cuando se crea la instancia de Jugador Argentino y la instancia de Jugador de Conmebol.



8. Cobertura

Se logro obtener una buena cobertura de pruebas cercanas al 80 porciento lo cual permitio eliminar gran parte de las mutaciones presentes.

Not Covered Code (84% Code Coverage) [3]

AlgoConsejo class	valorDe:	not defined category	TP1
Valor	compararCon:precision:	not defined category	TP1
Valor	compararConValorEnReales:precision:	not defined category	TP1

Filter...

Current image: valorDe: x

Flat Hier. | Browse Class refs. Implementors Senders

```
valorDe: unNombreFutbolista
|unFutbolista valor|
valor := Valor new.
unFutbolista := futbolistas detect: [ :f | (f dameTuNombre) = unNombreFutbolista ].
valor asignaResultado: (unFutbolista dameTuValorEnPesos).
^ valor
```

1/7 [1] not defined category extension F +L W